

2.4.1 Sorting Algorithms: Bubble Sort

ببل سارٹ (Bubble Sort) میں ساتھ ساتھ موجود دو عناصر کا موازنہ کیا جاتا ہے اور اگر وہ غلط ترتیب میں ہوں تو ان کو...

CS XI

Chapter 2: Computational Thinking & Algorithm

Lecture 08 of 13

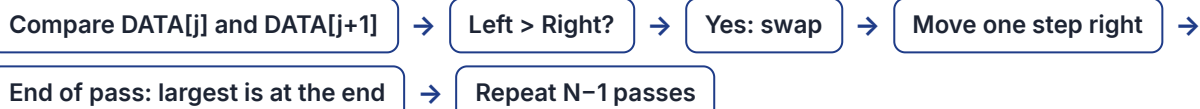
★ Golden

2.4.1 Sorting Algorithms: Bubble Sort ★ Golden

LEARN IT

Bubble Sort is an elementary sorting algorithm that works by repeatedly comparing **adjacent** (side-by-side) elements and swapping them if they are in the wrong order.

In each pass, the largest remaining value "bubbles up" to the end of the list, like an air bubble rising to the top of water. So after pass 1 the largest value is in the last place, after pass 2 the second largest is in second-last place, and so on. A list of N items needs **$N - 1$ passes**, and pass K makes **$N - K$ comparisons**.



Algorithm (textbook)

```

Step 1: Start
Step 2: Set DATA := {8,4,1,9,3}
Step 3: Set N := 5
Step 4: Repeat Step 5 and 6 for
        K = 1 to N-1
Step 5: Set PTR := 1
Step 6: Repeat while PTR ≤ N-K:
    a: If DATA[PTR] > DATA[PTR+1]:
        1. Set temp = DATA[PTR]
        2. Set DATA[PTR] = DATA[PTR+1]
        3. Set DATA[PTR+1] = temp
    b: Set PTR := PTR + 1
Step 7: End
  
```

Pseudocode

```

Start
DATA = {8,4,1,9,3}
N = 5
For outer = 0 to N-1
    For inner = 0 to N-2-outer
        IF DATA[inner] > DATA[inner+1]
            Temp = DATA[inner]
            DATA[inner] = DATA[inner+1]
            DATA[inner+1] = Temp
        End If
    End For
End For
End
  
```

WORKED EXAMPLE

Worked example: sort 8, 4, 1, 9, 3 in ascending order. The pair in brackets [] is being compared. Positions are counted from 1.

Pass 1 (K = 1, 4 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[8, 4], 1, 9, 3	8 > 4 Yes	Swap	4, 8, 1, 9, 3
2nd vs 3rd	4, [8, 1], 9, 3	8 > 1 Yes	Swap	4, 1, 8, 9, 3
3rd vs 4th	4, 1, [8, 9], 3	8 > 9 No	No change	4, 1, 8, 9, 3
4th vs 5th	4, 1, 8, [9, 3]	9 > 3 Yes	Swap	4, 1, 8, 3, 9

9 is now at its correct (final) place.

Pass 2 (K = 2, 3 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[4, 1], 8, 3, 9	4 > 1 Yes	Swap	1, 4, 8, 3, 9
2nd vs 3rd	1, [4, 8], 3, 9	4 > 8 No	No change	1, 4, 8, 3, 9
3rd vs 4th	1, 4, [8, 3], 9	8 > 3 Yes	Swap	1, 4, 3, 8, 9

Pass 3 (K = 3, 2 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[1, 4], 3, 8, 9	1 > 4 No	No change	1, 4, 3, 8, 9
2nd vs 3rd	1, [4, 3], 8, 9	4 > 3 Yes	Swap	1, 3, 4, 8, 9

Pass 4 (K = 4, 1 comparison)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[1, 3], 4, 8, 9	1 > 3 No	No change	1, 3, 4, 8, 9

Sorted list: 1, 3, 4, 8, 9. Totals: 4 + 3 + 2 + 1 = **10 comparisons** and **6 swaps**.

Pass	List after pass	Fixed at the end
Start	8, 4, 1, 9, 3	—
1	4, 1, 8, 3, 9	9
2	1, 4, 3, 8, 9	8, 9
3	1, 3, 4, 8, 9	4, 8, 9
4	1, 3, 4, 8, 9	all

WORKED EXAMPLE

Bubble Sort in Python (follows the pseudocode exactly; Python indexes start at 0):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(n - 1 - outer):
        if data[inner] > data[inner + 1]:
            temp = data[inner]
            data[inner] = data[inner + 1]
            data[inner + 1] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

OUTPUT

```
After pass 1 : [4, 1, 8, 3, 9]
After pass 2 : [1, 4, 3, 8, 9]
After pass 3 : [1, 3, 4, 8, 9]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]
```

`range(n - 1 - outer)` gives `inner = 0 ... N-2-outer`, exactly as in the pseudocode. Notice the list was already sorted after pass 3, but the plain algorithm still does pass 4.

Advantages and disadvantages

Advantages	Disadvantages
Easiest algorithm to visualise and explain.	Uses nested loops, so the time grows very fast as the list grows (about N^2 comparisons).
Does not need extra memory (in-place sorting).	Performs a massive number of swaps.
Stable (equal items stay in their original order).	Useless for real-world "Big Data".

REMEMBER IT

Bubble = neighbours. Bubble Sort only ever compares *neighbours* (j and $j+1$). Each pass pushes the biggest bubble to the end. For **descending** order, just change $>$ to $<$ in the comparison.

COMMON MISTAKE

The slides say the time grows "exponentially". Strictly, Bubble Sort's time grows **quadratically**: for N items it makes $N(N-1)/2$ comparisons, which is $O(N^2)$. Doubling the list makes it about 4 times slower — bad, but not exponential. Another common mistake: students forget that pass K does only $N - K$ comparisons; the sorted tail is never compared again.

اردو میں سمجھیں

ببل سارٹ (Bubble Sort) میں ساتھ ساتھ موجود دو عناصر کا موازنہ کیا جاتا ہے اور اگر وہ غلط ترتیب میں ہوں تو ان کی جگہ بدل دی جاتی ہے۔ ہر پاس (Pass) کے بعد سب سے بڑا عدد آخر میں پہنچ جاتا ہے۔ N عناصر کے لیے $N-1$ پاس درکار ہوتے ہیں۔ یہ سمجھنے میں آسان اور مستحکم (Stable) ہے لیکن بڑے ڈیٹا کے لیے بہت سست ہے۔

EXAM TIP

The most common question: "Sort the list ... using Bubble Sort. Show each pass." Draw one table per pass with columns *Pair*, *Compare*, *Action*, *List after*. Mark the element fixed at the end after every pass. Also be ready to write the algorithm (with PTR and K) or the pseudocode (with outer/inner), and to list two advantages and two disadvantages.

CHECK YOURSELF

1. How many passes and comparisons does Bubble Sort need for 6 elements?
2. Show the list 5, 1, 4 after pass 1 of Bubble Sort.
3. Sort 2, 9, 4, 7 in descending order with Bubble Sort: give the list after each pass.

Answers: 1. 5 passes; $5 + 4 + 3 + 2 + 1 = 15$ comparisons. 2. (5,1) swap $\rightarrow$ 1, 5, 4; (5,4) swap $\rightarrow$ 1, 4, 5. 3. Pass 1: 9, 4, 7, 2; Pass 2: 9, 7, 4, 2; Pass 3: 9, 7, 4, 2.

Next lecture: 2.4.1 Sorting Algorithms: Selection Sort